Sistem Mimarisini İfade Eden UML Yapı Diyagramları

Sınıf Diyagramı

Bileşen Diyagramı

Barındırma Diyagramı

Paket Diyagramı

Davranışsal Diyagramlar

Davranışsal Diyagramlar

Yapısal diyagramlar sistemi bir bütün olarak tanımlarken, davranışsal diyagramlar belirli bir davranış örneği ile ilgilidir.

Genel sistem davranışını göstermek için birden çok ardışıl veya işbirliği diyagramına ihtiyaç vardır.

Bir sistemde tüm diyagram tiplerini yine kullanmak zorunda değiliz.

Kullanım Senaryosu (Use Case) Diyagramı

UML'de, OMT'nin veri akış diyagramları yerine Jacobson'un kullanım senaryosu diyagramları yer alır.

Kullanım senaryosu: Sistem tepkilerini de içerecek şekilde kullanıcı tarafından gözlenebilir eylemler sırası.

Sistemin belirli bir kullanıcı etkileşimi ile nasıl başa çıktığının hikayesidir.

Kullanım senaryoları özellikle gereksinimleri ortaya çıkarmak için yararlıdır.

Sistemin nasıl kullanılacağına dair değişik hikayeleri ortaya çıkarıp, sonra dallanmaları somutlaştırırız.

- Birşeyler yanlış olursa ne olacak?
- Belirgin yapmadığımız ara adımlarda neler olacak?

Kullanım Senaryosu (Use Case) Diyagramı

İki ana parçası vardır

Çöp adamlar dış aktörleri ifade eder. Sistemin kullanıcıları oldukları gibi, başka sistemler ya da harici cihazlar da olabilirler.

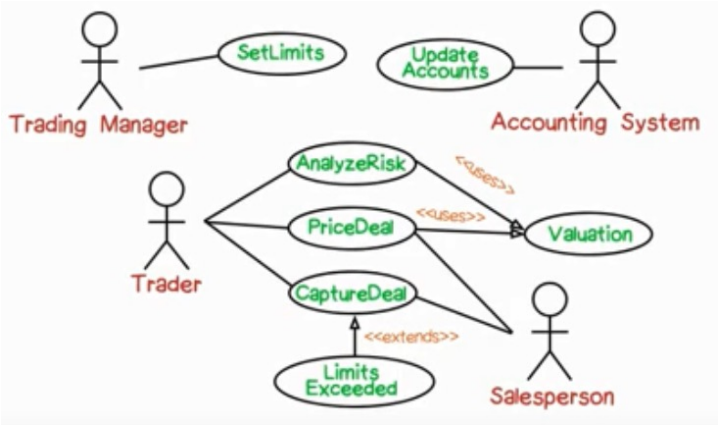
Elipsler senaryolardır. Kullanım senaryosu diyagramı, bir sistem hikayesi değildir. Sistem senaryoları kümesinin tanımıdır.

Açıklama taşımayan çizgiler ortaklık/katılım demektir. Bir uçtaki aktör, diğer uçtaki senaryo ile ilgilidir.

İki özel açıklama vardır.

- <<extends>> Bir hikayemiz var ve bunu zorunlu olmadan duruma göre değişen durumlarla genişletmek istiyoruz. *İkisi tek fiyatına.*
- <<include>> <<uses>> Alt yordam/fonksiyon çağrısı gibidir. Pek çok senaryoda ortak kullanılan davranış parçasıdır.

Kullanım Senaryosu Diyagramı



Kullanım Senaryosu

Diyagamda hepsi bir aradaydı.

Kullanım senaryosu bir hikayedir ve yapısız metin ya da tablo şeklinde yazılabilir.

Örneğin;

Ayşe Amazon'dan ürün almak ister.

Ayşe Amazon web sitesine girer.

Ayşe aradığı ürünü bulana kadar sitede gezer.

Ayşe ürünü sepete ekler.

Ayşe Satın alma sayfasına geçer.

Ayşe fatura bilgilerini verir ve ödemeyi tamamlar.

Kullanım Senaryosu Tablo Gösterimi

Aktör	Eylem	Nesne
Ayşe	gezer	Amazon websitesi
Ayşe	seçer	Kitap
Ayşe	ekler	Kitap, Sepet
Ayşe	Satın alır	
Amazon	ister	Fatura bilgisi
Ayşe	sağlar	Ad, adres, kart bilgisi
Amazon	doğrular	Kart bilgisi
Ayşe	onaylar	satış

Bağlam (Context) Diyagramı

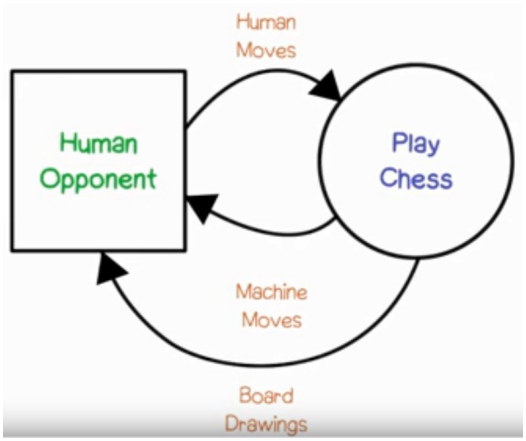
UML'in parçası değil ama ilginç kabiliyet sunar

En üst seviye veri akış diyagramıdır.

Tek bir elips içerir. Sistem bir bütün olarak yer alır.

Dikdörtgenler aktörleri gösterir.

Bağlantılar aralarındaki veri akışlarıdır.



Ardışıl (Sequence) Diyagram

Bir senaryoyu açıklar.

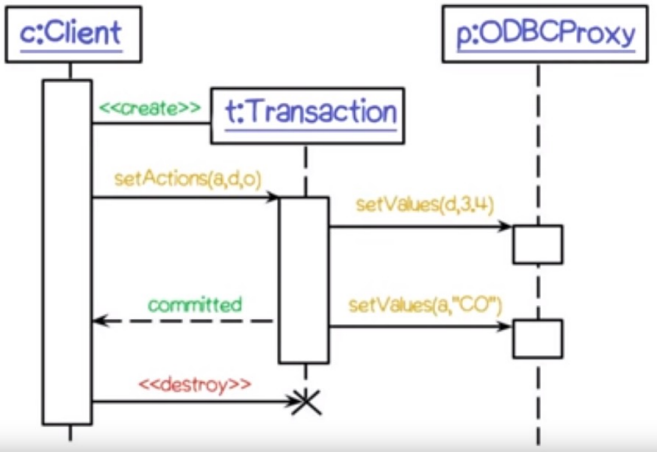
Her kolonda tekil bir katılımcı (genellikle nesneler) yer alır.

Zaman, aşağı doğru çizisel olarak devam eder.

Yatay çizgiler, nesneler arası mesajları gösterir.

Telefon sanayisindeki mesaj sırası kartlarından evrimleşmiştir.

İletişim diyagramlarıyla birebir aynı içeriğe sahiptir.



İletişim Diyagramı

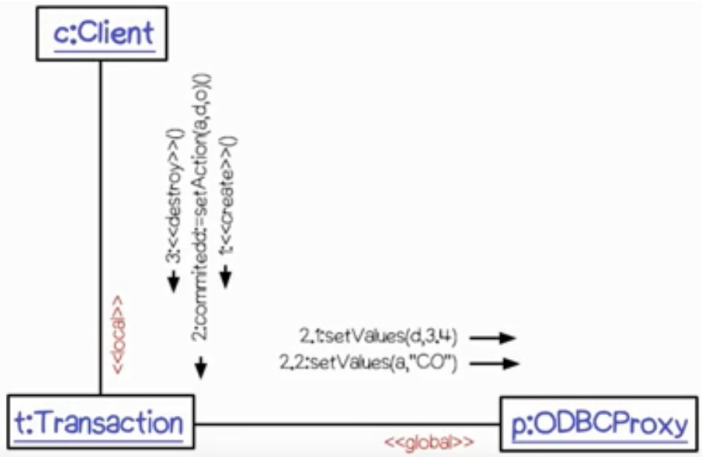
Sınıf diyagramına benzer.

Fark olarak çizgiler işlem çağrılarını (haberleşmeleri) taşır.

Bir önceki örneğin aynısı:

Mesaj belirteçleri numaralıdır. Sırayı gösterir.

Dewey ondalık gösterimini kullanır.
- 1, 2, 2.1, 2.2, 3, …



Aktivite Diyagramı

Ardışıl ve işbirliği diyagramlarında senkronizasyon bilgisi yer almaz.

Birden çok durumun aktif olabileceği bir durum geçiş makinesi betimler.
- Hepsinin kendi kontrol parçacığı vardır.

Petri ağlarından türetilmiştir.
- Petri ağlarında durum geçişleri aktivitenin tamamlanması ile tetiklenir.
- Dış olaylar yerine, durumu bitirmeniz gerekir.

Aktivite diyagramları ile iş akışlarını, işlem senkronizasyonunu ve eşzamanlılığı modelleyebiliriz.

Aktivite Diyagramı

Diyagramda tüm durumların üzerinde durabilecek bir simgemiz olsun.

Dolu daire olan tepeden başlar, yatay çizgiden ilerleyerek ilk duruma geçer. Sonra dörtgene geçer. Burada ikiye ayrılır. Biri sağa, biri de aşağı ilerler.

Sağdaki gidebildiği kadar aşağı (alttaki dörtgene kadar) gider.

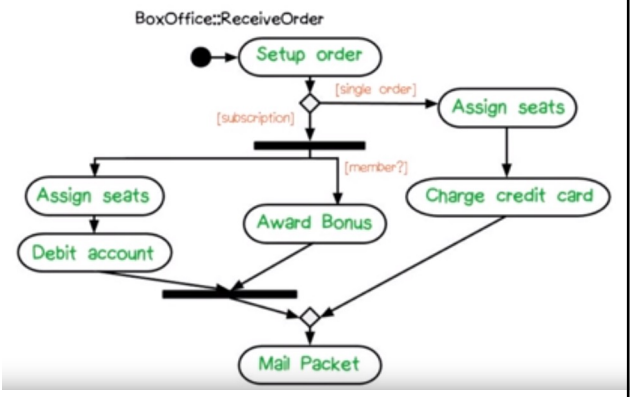
Aşağı giden, yatay siyah bar tarafından engellenir. Burası bir senkronizasyon noktasıdır.

Bu vakada senkronize edecek birşey yoktur ama altta iki çizgi çıkmaktadır. Herbirinde simgenin bir kopyası olur. Biri soldaki iki aktiviteye girer, ikincisi sağdan aşağı devam eder.

Sonunda bu ikisi ikinci yatay çizgide birleşir. Burası da bir senkronizasyon noktasıdır.

Kendi simgesine sahip iki yol bağımsız çalışır ve alttaki yatay çizgi sadece tepeden gelen iki simge de gelip birleştiğinde açılan bir kapı gibi davranır (Aktiviteleri senkronize eder).

Birleşen simge dörtgene (birleşme noktasına) girer, oradan da son duruma ve bitişe geçer.

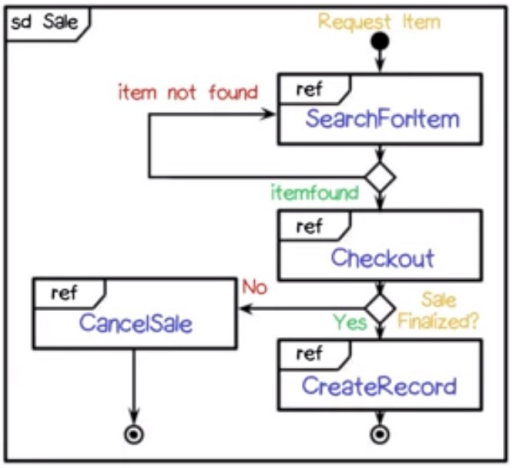


Genel Etkileşim Diyagramı

Bir çeşit aktivite diyagramıdır.

Düğümler alt seviyeli etkileşim diyagramlarını gösterir.

- Ardışıl, iletişim, genel etkileşim, zamanlama diyagramı olabilir.



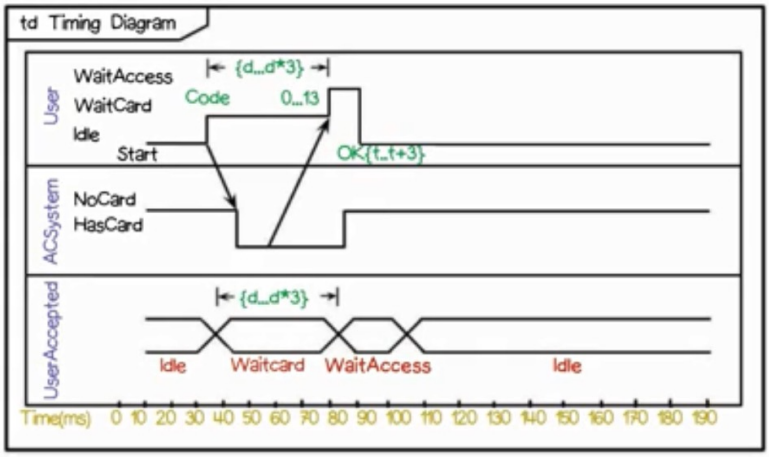
Zamanlama (Timing) Diyagramı

Dijital çip tasarımından miras

Sistemde durumların belirli zamanlamaları olması gerektiğinde kullanılır.

Zaman soldan sağa ilerler,

Oklar zamanlama senkronizasyon noktalarını gösterir.



Durum (state) Diyagramı

Davranışsal diyagramların en güçlü ve karmaşık olanı

Sonlu durum makinelerinin yeteneklerini, birleştirme (aggregation), eşzamanlılık, geçmiş, olayları tüme yayım gibi maddelerle genişletir.

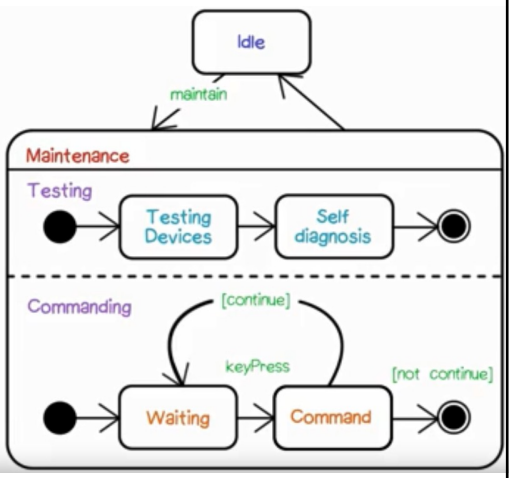
Bunlarla ilgili bir dersimiz olacak.

Durum Diyagramı

İki harici durum: Boş ve Bakım

Bakım durumunda alt durumlar var.
- Test
- Komut

Bunlar eşzamanlı çalışır.
- Test durumu soldan sağa çalışır
- Komut durumu, son duruma gelene kadar pekçok kez çalışabilir.



UML Davranış Diyagramları - Genel

Aktivite: Aktiviteler arası akış kontrolü

Ardışıl: Mesajlaşan sınıflar arası etkileşim

İletişim: Numaralandırılmış mesajlarla nesne etkileşimi

Genel Etkileşim: Daha alt seviyedeki aktivite diyagramlarının sentezi

Zamanlama: Döndürülmüş ardışıl diyagram

Kullanım Senaryosu: Harici aktörlere sağlanan sistem fonksiyonları

Durum: Uyarana tepki olarak dinamik davranış

Tekrar: Geliştireceğiniz bir sistemde hepsine ihtiyaç duymazsınız. En popüler olanlarını kullanırsınız.

İhtiyacınız olursa, hepsi ortada duruyor.

Nesne Kısıt Dili

UML'in diyagramsız kısmı ve diyagramların metin uzantısı

Diyagramlarda tanımlayamadığımız şeyler için kullanılır.

- Sınıf ve durum geçiş diyagramlarında metin uzantısı olur.

Sistem tanımlarında kesin olma gereği olduğunda kullanılır.

Nesne Kısıt Dili

Sınıf diyagramında bir hesap sınıfı var. Hesap sınıfının para yatırma işlemi olsun.

Bu işlem, "miktar" isimli bir gerçek sayı alsın.

Pre anahtar kelimesi miktarın sıfırdan büyük olmasını gösterir.

Post anahtar kelimesi son miktarın, mevcut + ilk miktar olmasını gösterir.

```
context Account::deposit(Real : amount)
pre: amount > 0
post: balance = balance@pre + amount
```

UML Meta Model

UML'in UML ile tanımı

UML meta modeli UML profilleri ile Profil diyagramı kullanarak genişletebilirsiniz.

Yapacağınız eklentiler ilave edeceğiniz stereotipler, etiket değerleri ve kısıtlardır.

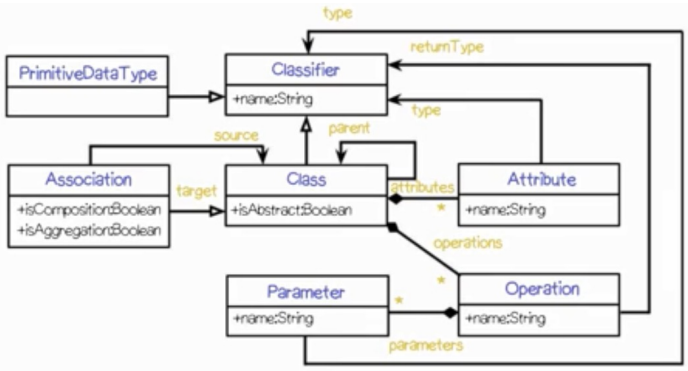
Kısıtlar modelde değil diyagramlardadır.

UML Meta Model

Kendi sınıfı olanlar:

- Sınıflar
- Sınıfların özellikleri
- Bir ilişki sınıfı
- Sınıflar arası ilişkiler

UML'i oluşturan çeşitli parçaların hepsi burada birarada bulunuyor.



Sonuç

Karmaşık bir sistemi ne yapacağını varsaymadan tasarlayamazsınız.

Diyagramlar problemi nasıl anladığınızı ve çözümünüzü nasıl ifade ettiğinizi temsil eder.

UML'de kullanabileceğiniz pek çok diyagram vardır, bunların yanında OCL ve meta model de mevcuttur.

Problemi ne kadar iyi anlarsanız, sistemin geliştirme geçmişinde o kadar az alt problemle karşılaşırsınız.

Nesneye Yönelik Analiz